Superior University
Department Of Software Engineering

Project Name :

Car Rental System

Subject :

Object Oriented Programming

Section :

BSSE-2B

Name : Muhummad Sami Abid **Roll No :** SU92-BSSEM-F24-117
Name : Muhummad Ahsan Iqbal **Roll No :** SU92-BSSEM-F24-134
Name : Abdul Rehman Gallani **Roll No :** SU92-BSSEM-F24-100
Name : Muhummad Asim Yameen **Roll No :** SU92-BSSEM-F24-115

Submitted To :

Ma'am Ayesha Karim

Diagram 1:

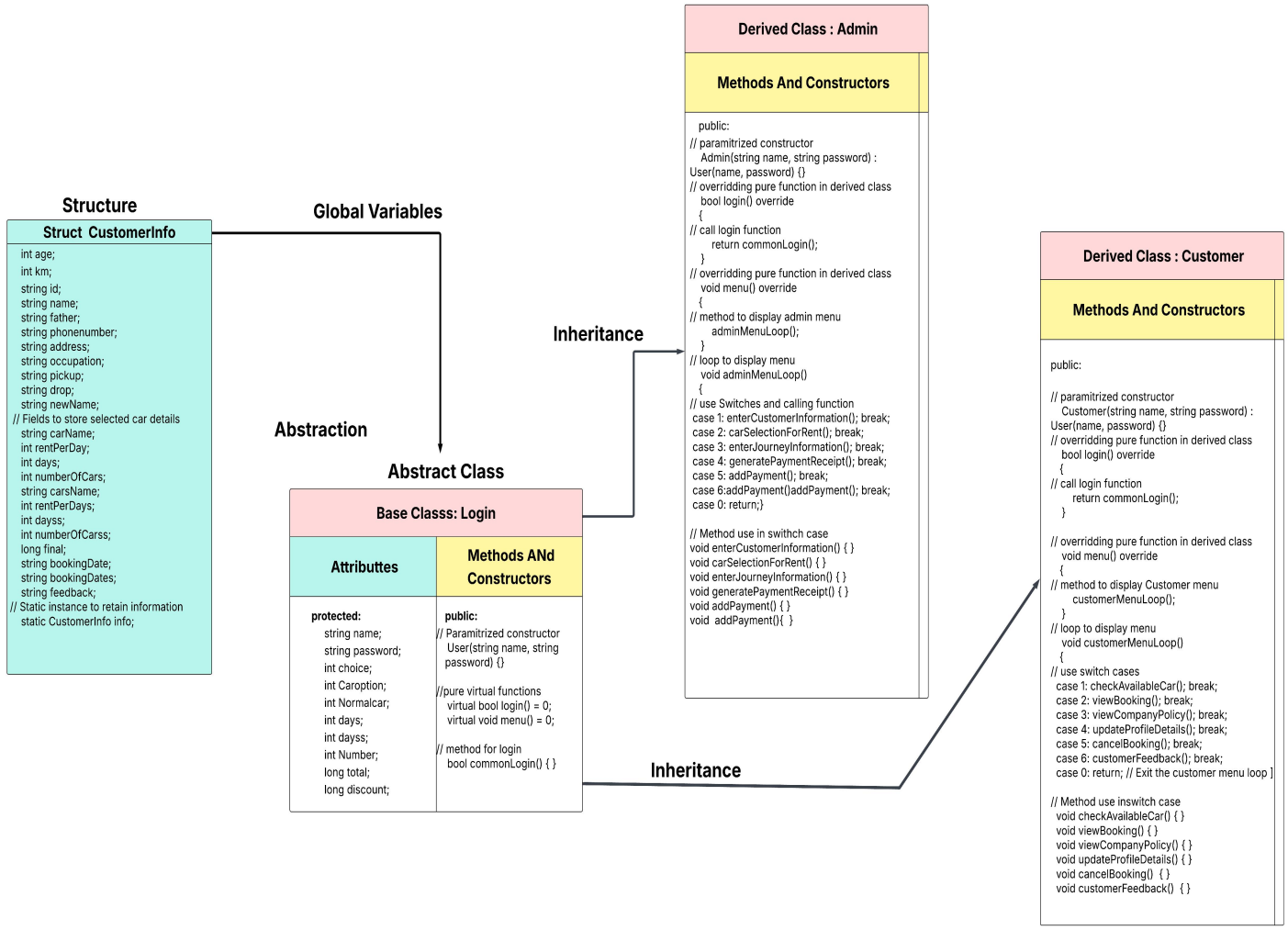

Structure
Struct CustomerInfo
int age;
int km;
string id;
string name;
string father;
string phonenumber;
string address;
string occupation;
string pickup;
string drop;
string newName;
// Fields to store selected car details
string carName;
int rentPerDay;
int days;
int numberOfCars;
string carsName;
int rentPerDays;
int dayss;
int numberOfCarss;
long final;
string bookingDate;
string bookingDates;
string feedback;
// Static instance to retain information
static CustomerInfo info;
Global Variables
Abstraction
Abstract Class
Base Classs: Login
Attributtes
Methods ANd Constructors
protected:
string name;
string password;
int choice;
int Caroption;
int Normalcar;
int days;
int dayss;
int Number;
long total;
long discount;
public:
// Paramitrized constructor
User(string name, string password) {}
//pure virtual functions
virtual bool login() = 0;
virtual void menu() = 0;
// method for login
bool commonLogin() { }
Inheritance
Derived Class : Admin
Methods And Constructors
public:
// paramitrized constructor
Admin(string name, string password) :
User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display admin menu
adminMenuLoop();
}
// loop to display menu
void adminMenuLoop()
{
// use Switches and calling function
case 1: enterCustomerInformation(); break;
case 2: carSelectionForRent(); break;
case 3: enterJourneyInformation(); break;
case 4: generatePaymentReceipt(); break;
case 5: addPayment(); break;
case 6:addPayment()addPayment(); break;
case 0: return;}
// Method use in swithch case
void enterCustomerInformation() { }
void carSelectionForRent() { }
void enterJourneyInformation() { }
void generatePaymentReceipt() { }
void addPayment() { }
void addPayment(){ }
Inheritance
Derived Class : Customer
Methods And Constructors
public:
// paramitrized constructor
Customer(string name, string password) :
User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display Customer menu
customerMenuLoop();
}
// loop to display menu
void customerMenuLoop()
{
// use switch cases
case 1: checkAvailableCar(); break;
case 2: viewBooking(); break;
case 3: viewCompanyPolicy(); break;
case 4: updateProfileDetails(); break;
case 5: cancelBooking(); break;
case 6: customerFeedback(); break;
case 0: return; // Exit the customer menu loop }
// Method use inswitch case
void checkAvailableCar() { }
void viewBooking() { }
void viewCompanyPolicy() { }
void updateProfileDetails() { }
void cancelBooking() { }
void customerFeedback() { }

Diagram 2:

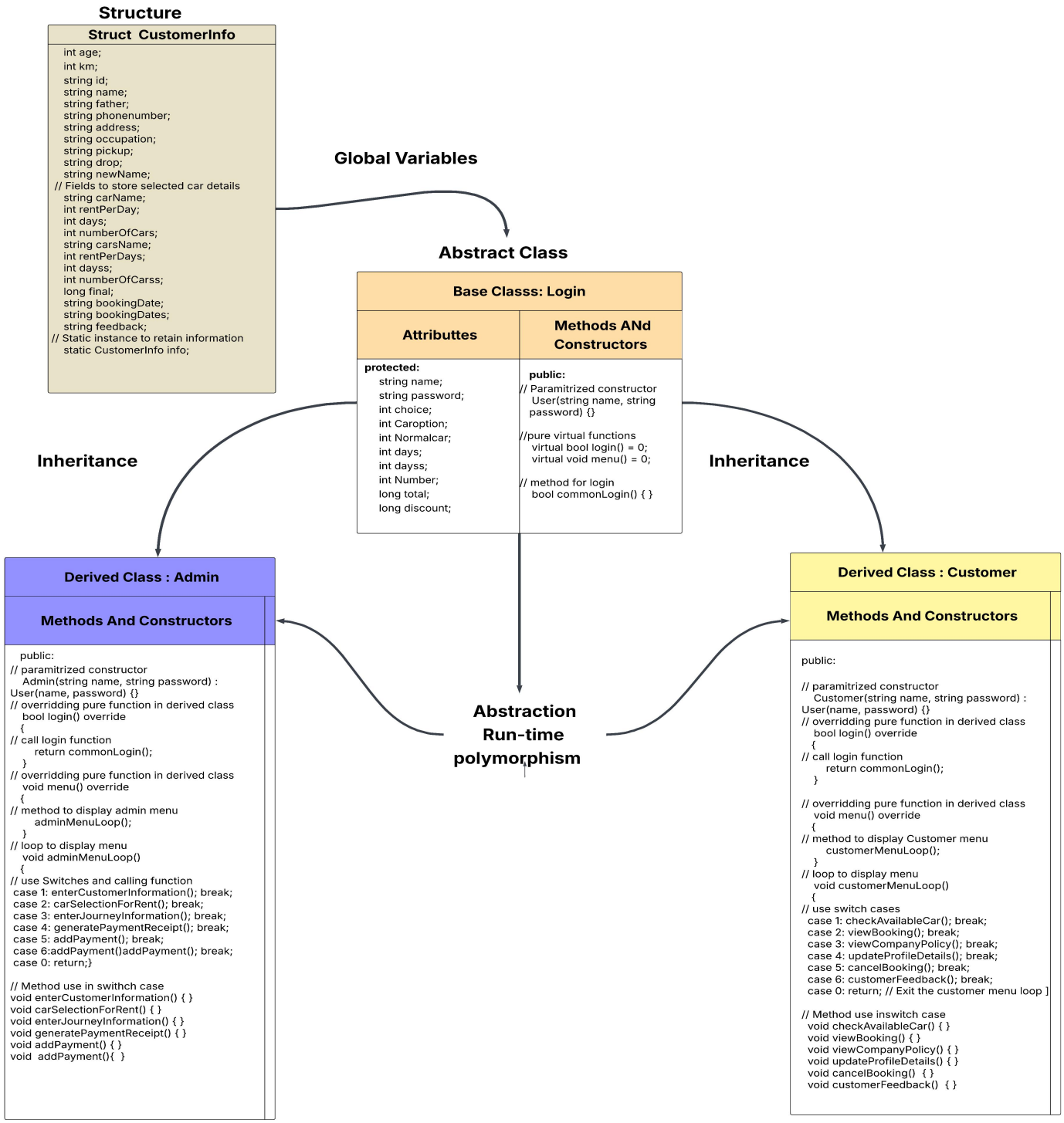

Structure
Struct CustomerInfo
int age;
int km;
string id;
string name;
string father;
string phonenumber;
string address;
string occupation;
string pickup;
string drop;
string newName;
// Fields to store selected car details
string carName;
int rentPerDay;
int days;
int numberOfCars;
string carsName;
int rentPerDays;
int dayss;
int numberOfCarss;
long final;
string bookingDate;
string bookingDates;
string feedback;
// Static instance to retain information
static CustomerInfo info;
Global Variables
Abstract Class
Base Classs: Login
Attributtes
Methods ANd Constructors
protected:
string name;
string password;
int choice;
int Caroption;
int Normalcar;
int days;
int dayss;
int Number;
long total;
long discount;
public:
// Paramitrized constructor
User(string name, string password) {}
//pure virtual functions
virtual bool login() = 0;
virtual void menu() = 0;
// method for login
bool commonLogin() { }
Inheritance
Inheritance
Derived Class : Admin
Methods And Constructors
public:
// paramitrized constructor
Admin(string name, string password) : User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display admin menu
adminMenuLoop();
}
// loop to display menu
void adminMenuLoop()
{
// use Switches and calling function
case 1: enterCustomerInformation(); break;
case 2: carSelectionForRent(); break;
case 3: enterJourneyInformation(); break;
case 4: generatePaymentReceipt(); break;
case 5: addPayment(); break;
case 6:addPayment()addPayment(); break;
case 0: return;}
// Method use in swithch case
void enterCustomerInformation() { }
void carSelectionForRent() { }
void enterJourneyInformation() { }
void generatePaymentReceipt() { }
void addPayment() { }
void addPayment(){ }
Abstraction
Run-time
polymorphism
Derived Class : Customer
Methods And Constructors
public:
// paramitrized constructor
Customer(string name, string password) : User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display Customer menu
customerMenuLoop();
}
// loop to display menu
void customerMenuLoop()
{
// use switch cases
case 1: checkAvailableCar(); break;
case 2: viewBooking(); break;
case 3: viewCompanyPolicy(); break;
case 4: updateProfileDetails(); break;
case 5: cancelBooking(); break;
case 6: customerFeedback(); break;
case 0: return; // Exit the customer menu loop]
// Method use inswitch case
void checkAvailableCar() { }
void viewBooking() { }
void viewCompanyPolicy() { }
void updateProfileDetails() { }
void cancelBooking() { }
void customerFeedback() { }

Diagram 3:

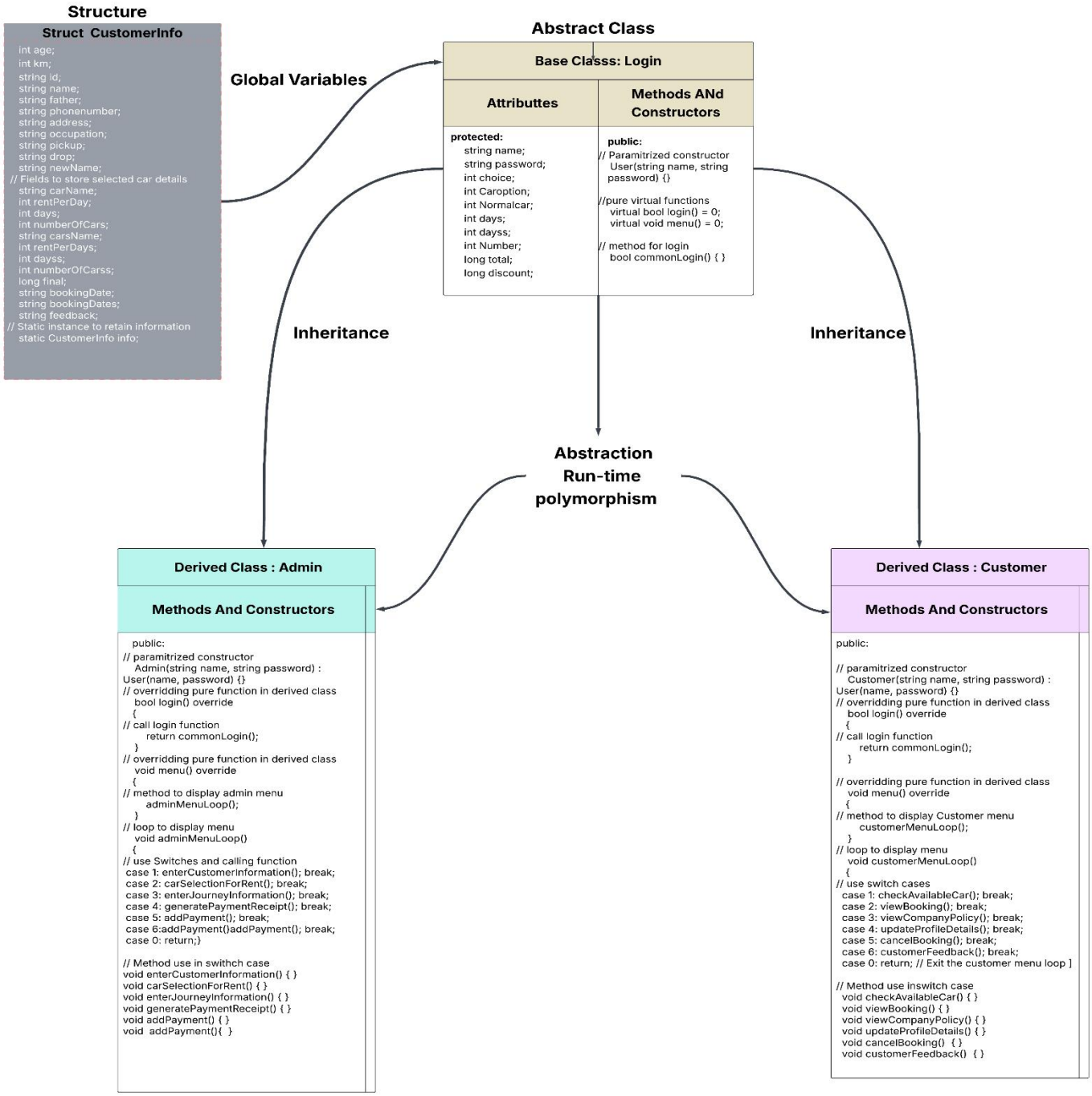

Structure
Struct CustomerInfo
int age;
int km;
string id;
string name;
string father;
string phonenumber;
string address;
string occupation;
string pickup;
string drop;
string newName;
// Fields to store selected car details
string carName;
int rentPerDay;
int days;
int numberOfCars;
string carsName;
int rentPerDays;
int dayss;
int numberOfCarss;
long final;
string bookingDate;
string bookingDates;
string feedback;
// Static instance to retain information
static CustomerInfo info;
Global Variables
Abstract Class
Base Classs: Login
Attributtes
Methods ANd Constructors
protected:
string name;
string password;
int choice;
int Caroption;
int Normalcar;
int days;
int dayss;
int Number;
long total;
long discount;
public:
// Paramitrized constructor
User(string name, string password) {}
//pure virtual functions
virtual bool login() = 0;
virtual void menu() = 0;
// method for login
bool commonLogin() { }
Inheritance
Inheritance
Abstraction
Run-time
polymorphism
Derived Class : Admin
Methods And Constructors
public:
// paramitrized constructor
Admin(string name, string password) :
User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display admin menu
adminMenuLoop();
}
// loop to display menu
void adminMenuLoop()
{
// use Switches and calling function
case 1: enterCustomerInformation(); break;
case 2: carSelectionForRent(); break;
case 3: enterJourneyInformation(); break;
case 4: generatePaymentReceipt(); break;
case 5: addPayment(); break;
case 6:addPayment()addPayment(); break;
case 0: return;}
// Method use in swithch case
void enterCustomerInformation() { }
void carSelectionForRent() { }
void enterJourneyInformation() { }
void generatePaymentReceipt() { }
void addPayment() { }
void addPayment(){ }
Derived Class : Customer
Methods And Constructors
public:
// paramitrized constructor
Customer(string name, string password) :
User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display Customer menu
customerMenuLoop();
}
// loop to display menu
void customerMenuLoop()
{
// use switch cases
case 1: checkAvailableCar(); break;
case 2: viewBooking(); break;
case 3: viewCompanyPolicy(); break;
case 4: updateProfileDetails(); break;
case 5: cancelBooking(); break;
case 6: customerFeedback(); break;
case 0: return; // Exit the customer menu loop]
// Method use inswitch case
void checkAvailableCar() { }
void viewBooking() { }
void viewCompanyPolicy() { }
void updateProfileDetails() { }
void cancelBooking() { }
void customerFeedback() { }

Diagram 4:

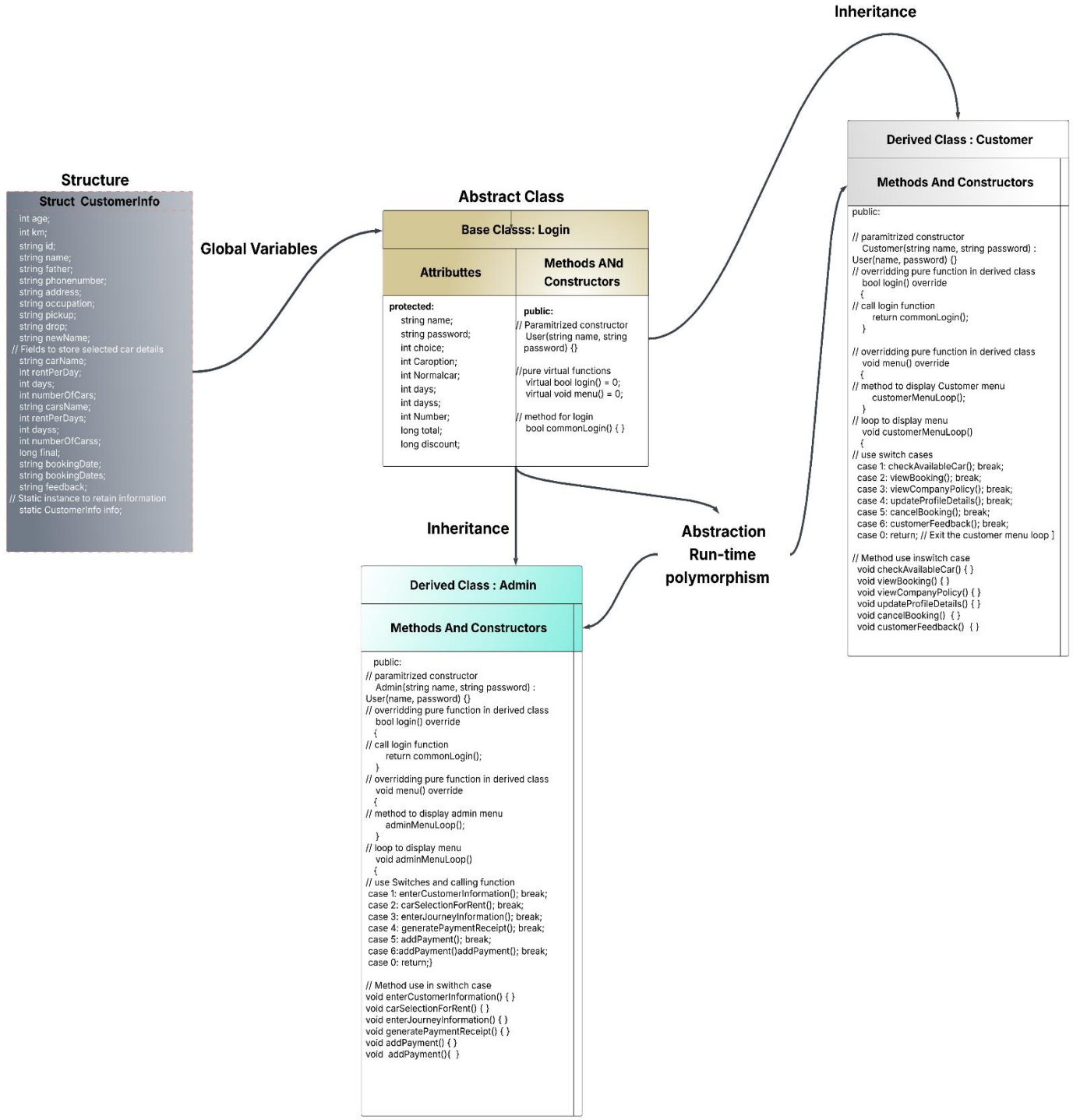
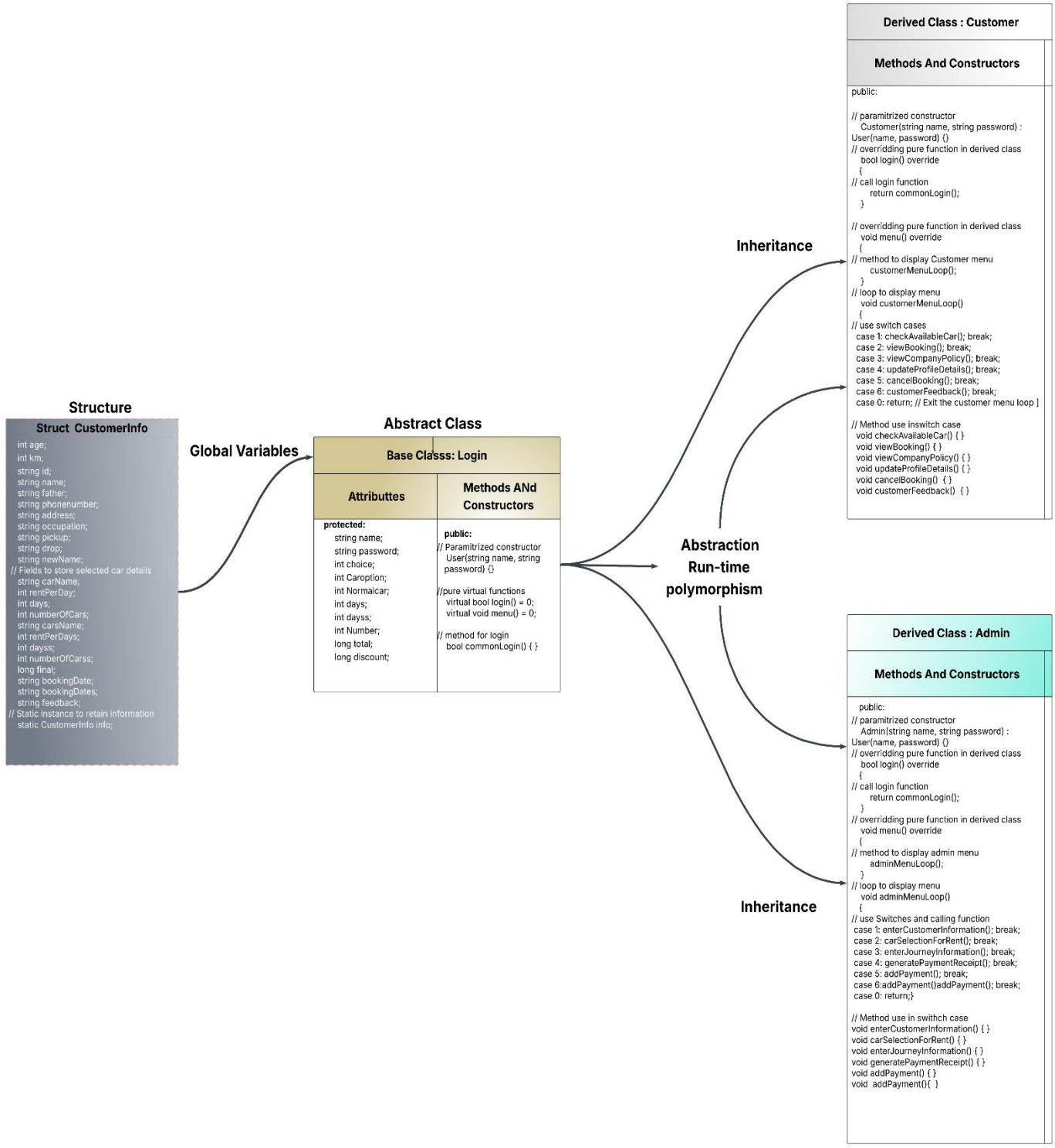

Inheritance
Structure
Struct CustomerInfo
int age;
int km;
string id;
string name;
string father;
string phonenumber;
string address;
string occupation;
string pickup;
string drop;
string newName;
// Fields to store selected car details
string carName;
int rentPerDay;
int days;
int numberOfCars;
string carsName;
int rentPerDays;
int dayss;
int numberOfCarss;
long final;
string bookingDate;
string bookingDates;
string feedback;
// Static instance to retain information
static CustomerInfo info;
Global Variables
Abstract Class
Base Classs: Login
Attributtes
Methods ANd Constructors
protected:
string name;
string password;
int choice;
int Caroption;
int Normalcar;
int days;
int dayss;
int Number;
long total;
long discount;
public:
// Paramitrized constructor
User(string name, string password) {}
//pure virtual functions
virtual bool login() = 0;
virtual void menu() = 0;
// method for login
bool commonLogin() { }
Derived Class : Customer
Methods And Constructors
public:
// paramitrized constructor
Customer(string name, string password) :
User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display Customer menu
customerMenuLoop();
}
// loop to display menu
void customerMenuLoop()
{
// use switch cases
case 1: checkAvailableCar(); break;
case 2: viewBooking(); break;
case 3: viewCompanyPolicy(); break;
case 4: updateProfileDetails(); break;
case 5: cancelBooking(); break;
case 6: customerFeedback(); break;
case 0: return; // Exit the customer menu loop]
// Method use inswitch case
void checkAvailableCar() { }
void viewBooking() { }
void viewCompanyPolicy() { }
void updateProfileDetails() { }
void cancelBooking() { }
void customerFeedback() { }
Inheritance
Abstraction
Run-time
polymorphism
Derived Class : Admin
Methods And Constructors
public:
// paramitrized constructor
Admin(string name, string password) :
User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display admin menu
adminMenuLoop();
}
// loop to display menu
void adminMenuLoop()
{
// use Switches and calling function
case 1: enterCustomerInformation(); break;
case 2: carSelectionForRent(); break;
case 3: enterJourneyInformation(); break;
case 4: generatePaymentReceipt(); break;
case 5: addPayment(); break;
case 6:addPayment()addPayment(); break;
case 0: return;}
// Method use in swithch case
void enterCustomerInformation() { }
void carSelectionForRent() { }
void enterJourneyInformation() { }
void generatePaymentReceipt() { }
void addPayment() { }
void addPayment(){ }

Diagram 5:


Structure
Struct CustomerInfo
int age;
int km;
string id;
string name;
string father;
string phonenumber;
string address;
string occupation;
string pickup;
string drop;
string newName;
// Fields to store selected car details
string carName;
int rentPerDay;
int days;
int numberOfCars;
string carsName;
int rentPerDays;
int dayss;
int numberOfCarss;
long final;
string bookingDate;
string bookingDates;
string feedback;
// Static instance to retain information
static CustomerInfo info;
Global Variables
Abstract Class
Base Classs: Login
Attributtes
Methods ANd Constructors
protected:
string name;
string password;
int choice;
int Caroption;
int Normalcar;
int days;
int dayss;
int Number;
long total;
long discount;
public:
// Paramitrized constructor
User(string name, string password) {}
//pure virtual functions
virtual bool login() = 0;
virtual void menu() = 0;
// method for login
bool commonLogin() { }
Inheritance
Abstraction
Run-time
polymorphism
Derived Class : Customer
Methods And Constructors
public:
// paramitrized constructor
Customer(string name, string password) :
User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display Customer menu
customerMenuLoop();
}
// loop to display menu
void customerMenuLoop()
{
// use switch cases
case 1: checkAvailableCar(); break;
case 2: viewBooking(); break;
case 3: viewCompanyPolicy(); break;
case 4: updateProfileDetails(); break;
case 5: cancelBooking(); break;
case 6: customerFeedback(); break;
case 0: return; // Exit the customer menu loop]
// Method use inswitch case
void checkAvailableCar() { }
void viewBooking() { }
void viewCompanyPolicy() { }
void updateProfileDetails() { }
void cancelBooking() { }
void customerFeedback() { }
Derived Class : Admin
Methods And Constructors
public:
// paramitrized constructor
Admin(string name, string password) :
User(name, password) {}
// overridding pure function in derived class
bool login() override
{
// call login function
return commonLogin();
}
// overridding pure function in derived class
void menu() override
{
// method to display admin menu
adminMenuLoop();
}
// loop to display menu
void adminMenuLoop()
{
// use Switches and calling function
case 1: enterCustomerInformation(); break;
case 2: carSelectionForRent(); break;
case 3: enterJourneyInformation(); break;
case 4: generatePaymentReceipt(); break;
case 5: addPayment(); break;
case 6:addPayment()addPayment(); break;
case 0: return;}
// Method use in swithch case
void enterCustomerInformation() { }
void carSelectionForRent() { }
void enterJourneyInformation() { }
void generatePaymentReceipt() { }
void addPayment() { }
void addPayment(){ }
Inheritance